Skill-R1: Agent Skill Evolution via Reinforcement Learning

Yash Vishe[1], **Rohan Surana**[1], **Xunyi Jiang**[1], **Zihan Huang**[1], **Xintong Li**[1], **Nikki Lijing Kuang**[1], **Tong Yu**[2], **Ryan A. Rossi**[2], **Jingbo Shang**[1], **Julian McAuley**[1], **Junda Wu**[1]
[1]UC San Diego [2]Adobe Research
{yvishe,rsurana,xuj003,zih043,xil240,jshang,jmcauley,juw069}@ucsd.edu
{tyu,ryan.rossi}@adobe.com

Abstract

Agentic large language models often rely on skills, reusable natural-language procedures that guide planning, action, and tool use. In practice, however, skills are typically improved through prompt engineering or by aligning the task LLM itself to revised skills, which is costly, model-specific, and often infeasible for closed-source models. Meanwhile, skill optimization is not a one-step prompt engineering problem, but a recurrent process with two coupled levels of credit assignment. A useful skill must improve rollout quality under the current conditioning, while a useful revision must turn observed successes and failures into a better skill for the next round. We therefore formulate skill evolution as a bi-level optimization problem over rollout selection within each generation and skill improvement across generations. We propose **Skill-R1**, a reinforcement learning framework for instance-level recurrent skill optimization from verifiable rewards. Rather than updating the task LLM, **Skill-R1** trains a lightweight skill generator that conditions on the task context, prior rollouts, and their verified outcomes to produce skills that steer a frozen task LLM. This design preserves black-box compatibility with both open- and closed-source models while making adaptation substantially cheaper than model-level updates. **Skill-R1** proceeds over multiple generations. At each generation, the current skill induces rollouts from the task LLM, whose verified outcomes are fed back to produce the next revision. To optimize the skill generator over this recurrent process, we introduce a bi-level group-relative policy optimization objective with *intra-generation* and *inter-generation* advantages. The intra-generation term compares rollouts under shared skill conditioning, while the inter-generation term rewards revisions that improve the induced behavior over successive generations. Together, these terms provide a principled objective for directional skill evolution rather than one-shot or heuristic self-refinement. Empirical evaluations suggest that Skill-R1 achieves consistent improvements over both no-skill baselines and standard GRPO across benchmarks with verifiable rewards. The gains are particularly strong on complex, multi-step tasks, where single-pass refinement methods struggle.

1 Introduction

Agentic large language models increasingly rely on external skills (Xu & Yan, 2026; Jiang et al., 2026c; Chen et al., 2026b; Nguyen et al., 2025; Huang et al., 2025b), which are reusable natural-language procedures that specify how an agent should decompose a task, invoke tools, and verify intermediate results. Since skills are designed as complementary modules of the task language model, they provide a practical interface for shaping LLM behavior without updating the task language model. This separation is practical in realistic deployments, where the task model may be proprietary (Gao et al., 2025; Huang et al., 2025a), expensive to adapt (Zhou et al., 2025; Wu et al., 2024a; 2025d; Wang et al., 2025c; Zhang et al., 2024), or shared across many downstream tasks (Li et al., 2026; Chen et al., 2026a; Jiang et al., 2026a).

However, many skill-based systems rely on well-engineered skills as predefined and fixed artifacts, or revise them only through prompting and single-round self-refinement (Jiang et al., 2026c; Xu & Yan, 2026; Chen et al., 2026b; Jiang et al., 2026a; Chen et al., 2026a; Xia et al., 2026; Zhang et al., 2026). These strategies can be misaligned and inefficient, since they rely on repeated prompt edits and are not directly aligned with downstream rewards (Wu et al.; Kveton et al., 2025; Xia et al., 2025a). On the other hand, some recent works adapt the task language model to the skill by fine-tuning the task language model on the skill (Wang et al., 2025a; Li et al., 2025b; Wang et al., 2024), but this approach can be computationally costly and limited in adaptivity to many downstream tasks (Zhou et al., 2025; Xia et al., 2026; Jiang et al., 2026b; Wu et al., 2025d; Yao et al., 2024). Moreover, skill optimization should not be a single prompt-editing step, but a recurrent decision problem: a useful skill should improve rollout quality under the current context, and a useful revision should transform observed successes and failures into a better skill for the next round. This naturally leads to a bi-level optimization objective that couples execution quality within each generation with skill improvement across generations.

To address this problem, we propose **Skill-R1**, a reinforcement learning framework for recurrent agent skill evolution from verifiable rewards. **Skill-R1** separates *task execution* from *skill improvement*. A frozen task LLM produces rollouts conditioned on the current skill, while a lightweight skill generator is trained to revise that skill based on the task context, prior rollouts, and their verified outcomes (Xia et al., 2025c; Wang et al., 2025b; Van Nguyen et al., 2025). By updating only the skill generator, the framework remains compatible with both open- and closed-source task models and can be more efficient than updating the task model itself (Van Nguyen et al., 2025; Xia et al., 2025a).

The core interaction loop of **Skill-R1** is multi-generation. For a given instance, the current skill induces a group of rollouts from the frozen task model, a verifier scores these rollouts, and the resulting trials and errors (Wu et al., 2025b; 2024b), and reward signals are appended to an instance-specific history that conditions the next skill revision. Recurring execution of this process yields an evolutionary view of skill improvement, in which each generation proposes a new skill, tests it through a rollout population, and passes forward evidence for subsequent refinement. This recurrent structure makes it possible to optimize directional improvement (Xia et al., 2026; Sun et al., 2025; Zhou et al., 2025) on the same instance rather than selecting among isolated one-shot attempts.

The policy optimization of the skill generator requires both intra-generation and inter-generation credit assignment to optimize the skill generator. To enable optimization of the recurrent skill generator, we introduce a bi-level group-relative policy optimization (Zhong et al., 2026; Shao et al., 2024; Mundada et al., 2026) objective that mirrors the recurrent structure above. The *intra-generation* term compares rollouts sampled under the same skill and captures relative execution quality within a generation. The *inter-generation* term measures whether a revised skill improves the average verified performance relative to the previous generation. Together, these signals provide a principled objective for rewarding both strong rollouts and beneficial skill revisions (Wu et al.; Surana et al., 2026; Li et al., 2025a; Huang et al., 2025c). Our experiments demonstrate that Skill-R1 consistently improves agent performance across diverse reasoning and tool-use benchmarks, outperforming both no-skill baselines and standard GRPO. We also show a clear performance gap between skill generations, which demonstrates the effectiveness of the recurrent skill evolution. We summarize the contributions of this work as follows:

- We formulate recurrent instance-level skill evolution as a bi-level group-relative policy optimization problem.
- We propose **Skill-R1**, a model-agnostic framework that trains a lightweight skill generator while keeping the task LLM frozen.
- We introduce a recurrent multi-generation rollout process together with a bi-level GRPO objective that combines *intra-* and *inter-generation* advantages.
- We evaluate **Skill-R1** on agent tasks with verifiable rewards, comparing with agent skill and GRPO baselines and achieving superior performance.

2 Preliminaries

2.1 Agent Skills

Recent LLM-agent systems commonly treat skills as reusable units of external procedural knowledge that can be retrieved, executed, and updated without modifying the underlying task model. Following this practice, let π_{task} denote a frozen task LLM, and let $\mathcal{S}$ be a space of explicit skill artifacts.

Definition 2.1 (Agent Skill)**.** An agent skill is a reusable procedural artifact $s \in \mathcal{S}$ that conditions the execution of the task model on an input instance. A skill may encode natural-language instructions, structured workflows, tool-use guidance, or other procedural constraints, while remaining external to the parameters of π_{task}.

Given an instance x and a skill s, the frozen task model induces a rollout distribution

$$\pi_{\text{task}}(\tau \mid x, s), \tag{1}$$

where τ denotes the resulting trajectory. In our setting, skills evolve recurrently on the same instance. We write $s_g \in \mathcal{S}$ for the skill used at generation g, and $\mathcal{H}_g$ for the instance-specific history available up to generation g, such as prior rollouts and their verified outcomes. These variables will be used to formalize skill evolution in later sections.

2.2 Group-Relative Policy Optimization

We briefly review Group-Relative Policy Optimization (GRPO) in the language-generation setting, where a policy $\pi_\theta(\cdot \mid x)$ defines a distribution over response rollouts y conditioned on a prompt or context $x \sim \mathcal{D}$. GRPO samples a group

$$\mathcal{G}(x) = \{y^{(i)}\}_{i=1}^{K}, \qquad y^{(i)} \overset{\text{i.i.d.}}{\sim} \pi_\theta(\cdot \mid x), \tag{2}$$

assigns each rollout a verifier score $r(y^{(i)})$, and defines the group-relative advantage

$$A\left(y^{(i)}; \mathcal{G}(x)\right) = r\left(y^{(i)}\right) - \frac{1}{K}\sum_{j=1}^{K} r\left(y^{(j)}\right). \tag{3}$$

The GRPO objective is

$$\mathcal{L}_{\text{GRPO}}(\theta) = \mathbb{E}_{x\sim\mathcal{D},\,\mathcal{G}(x)}\left[\sum_{i=1}^{K} A\left(y^{(i)}; \mathcal{G}(x)\right) \log \pi_\theta\left(y^{(i)} \mid x\right)\right]. \tag{4}$$

This group-centered objective promotes rollouts that outperform their peers under the same input context.

3 Skill-R1: Agent Skill Evolution via Reinforcement Learning

We propose **Skill-R1** (illustrated in Figure 1), a reinforcement learning framework that improves a frozen task model by recurrently refining external skills without updating the task model itself. **Skill-R1** decouples *task execution* from *skill improvement*: a frozen task LLM generates rollout trajectories conditioned on a selected skill, while a separate editor policy updates skills based on verified execution outcomes.

3.1 Problem Setup

Let $x \in \mathcal{X}$ denote a task instance, and let $\mathcal{B}_x \subseteq \mathcal{S}$ denote the skill bank for instance x. Following Section 2.1, for a selected skill $s_0 \in \mathcal{B}_x$, the frozen task model π_{task} induces a rollout distribution Equation (1) over trajectories $y^{(i)} \in \mathcal{G}(x, s_0)$ group sampled (in Equation (2)) under skill conditioning, while a verifier $f_X : \mathcal{G}(x, s_0) \to \mathbb{R}$ assigns each rollout a scalar reward $r^{(i)}$. We also maintain the generation-wise history

$$\mathcal{H}_0 = \{(x, s_0, y^{(i)}, r^{(i)})\}_{i=1}^{K}, \qquad y^{(i)} \sim \pi_{\text{task}}(\cdot \mid x, s_0), \qquad i = 1, \ldots, K. \tag{5}$$

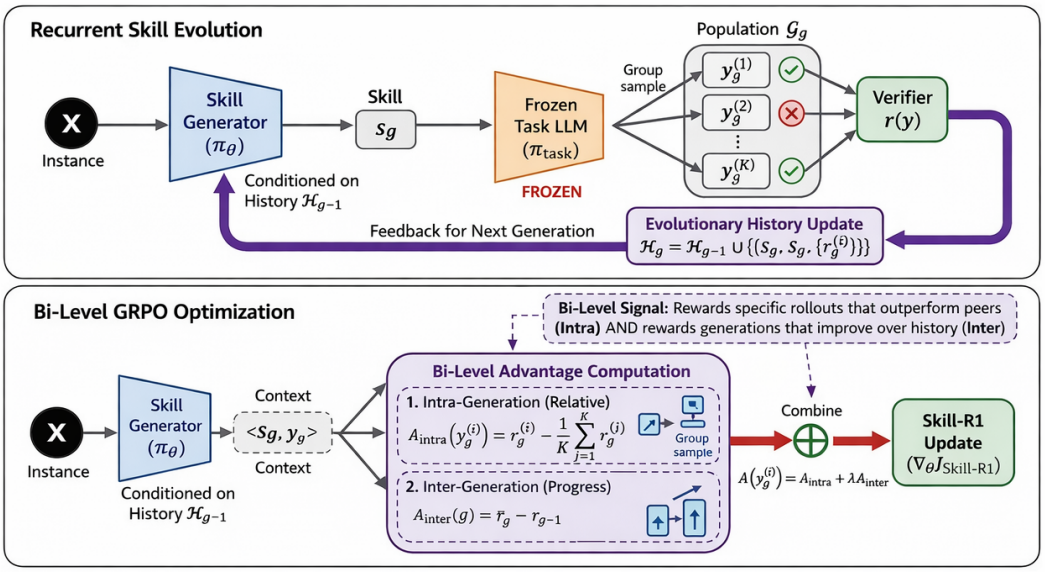

Figure 1: Overview of Skill-R1. **Recurrent Skill Evolution** iteratively generates skills, rolls out a frozen task LLM, and stores verifier-scored outcomes in an evolutionary history. **Bi-level GRPO Optimization** computes GRPO advantages from both intra-generation relative performance and inter-generation progress, and uses it to update the skill generator while keeping the task LLM frozen.

Thus, the instance-level performance induced by s_0 is determined by the rollout policy and verifier jointly, and our goal is to improve performance on the given instance by revising skills so that later rollout populations achieve higher verifier reward.

For each generation $g \geq 1$, a learnable skill generator π_θ produces the next skill conditioned on the current instance and the accumulated history:

$$\tau = \{(s_g, \mathcal{G}_g(x, s_g), \{r_g^{(i)}\}_{i=1}^{K})\}_{g=1}^{G}, \qquad s_g \sim \pi_\theta(\cdot \mid x, \mathcal{H}_{g-1}), \qquad g = 1, \ldots, G. \tag{6}$$

This defines a recurrent skill-evolution process in which each generated skill is evaluated through the rollout population it induces under the frozen task model Equation (1), and the resulting verifier outcomes are appended to the history to guide subsequent revisions. Then the recurrent skill evolution problem can be defined as optimizing π_θ to maximize verifier reward across the entire recurrent process:

$$J(\theta) = \mathbb{E}_{x \sim p(x)} \mathbb{E}_{s_g \sim \pi_\theta(\cdot|x, \mathcal{H}_{g-1}),\, y_g^{(i)} \sim \pi_{\text{task}}(\cdot|x, s_g)} \left[\sum_{g=1}^{G} \gamma^{g-1} \frac{1}{K} \sum_{i=1}^{K} r_g^{(i)} \right]. \tag{7}$$

Directly optimizing the objective in (7) is challenging because the reward signal is evaluated on the task model's rollouts rather than directly on the generated skill. To address this structural decoupling between skill generation and task execution, we extend the standard GRPO framework to a bi-level setting. In the following section, we formulate a bi-level GRPO objective and derive *bi-level advantages*. This formulation allows us to properly assign credit to the skill generator π_θ based on the relative, aggregate performance of the rollout populations that each skill induces.

3.2 Bi-Level Group-relative Advantages

The reward signal in Equation (7) is evaluated on rollouts from the frozen task model, yet the trainable parameters reside in the skill generator π_θ. To route rollout-level feedback to the skill that induced it, we decompose credit assignment into two complementary advantage signals. Within generation g, all K rollouts share the same skill s_g, so we apply

Algorithm 1 Skill-R1: Agent Skill Evolution via Reinforcement Learning

Require: Task distribution $p(x)$, frozen task LLM π_{task}, skill generator π_θ, initial skill bank $\mathcal{B}$, verifier f, generations G, group size K, mixing coefficient λ

for each task instance $x \sim p(x)$ and its skill bank $\mathcal{B}_x$ **do**
  Select initial skill $s_0 \in \mathcal{B}_x$
  Sample rollouts $\mathcal{G}_0(x, s_0) = \{y_0^{(i)}\}_{i=1}^K$ from $\pi_{\text{task}}(\cdot \mid x, s_0)$
  Score $r_0^{(i)} = f(y_0^{(i)})$ for each i; initialize $\mathcal{H}_0 = \{(x, s_0, y_0^{(i)}, r_0^{(i)})\}_{i=1}^K$
  for $g = 1, \ldots, G$ **do**
    Generate skill $s_g \sim \pi_\theta(\cdot \mid x, \mathcal{H}_{g-1})$ ▷ Equation (6)
    Sample rollouts $\mathcal{G}_g(x, s_g) = \{y_g^{(i)}\}_{i=1}^K$ from $\pi_{\text{task}}(\cdot \mid x, s_g)$
    Verify $r_g^{(i)} = f(y_g^{(i)})$ for each i
    Compute $A_{\text{intra}}(y_g^{(i)})$, $A_{\text{inter}}(g)$, and $A(y_g^{(i)})$ ▷ Equations (8) to (10)
    Update $\mathcal{H}_g = \mathcal{H}_{g-1} \cup \{(s_g, \mathcal{G}_g, \{r_g^{(i)}\}_{i=1}^K)\}$
  end for
end for
Update θ by maximizing $\mathcal{L}_{\text{Skill-R1}}(\theta)$ over accumulated rollouts ▷ Equation (11)

the group-relative comparison from Equation (3) directly:

$$A_{\text{intra}}(y_g^{(i)}) = r_g^{(i)} - \frac{1}{K}\sum_{j=1}^{K} r_g^{(j)}. \tag{8}$$

Centering by the group mean makes this signal invariant to absolute reward scale and isolates rollout quality from skill quality. However, A_{intra} is blind to whether a skill revision improved on its predecessor. To capture cross-generation progress, we define the population mean reward and the inter-generation advantage

$$A_{\text{inter}}(g) = \bar{r}_g - \bar{r}_{g-1}, \qquad \bar{r}_g = \frac{1}{K}\sum_{i=1}^{K} r_g^{(i)} \tag{9}$$

with $A_{\text{inter}}(1) = 0$. A positive value indicates that the revised skill shifted the rollout distribution toward higher reward; a negative value penalizes regressions. We combine the two signals into a single bi-level advantage:

$$A(y_g^{(i)}) = A_{\text{intra}}(y_g^{(i)}) + \lambda\, A_{\text{inter}}(g), \tag{10}$$

where $\lambda \geq 0$ interpolates between pure within-generation GRPO (λ=0) and a regime that additionally rewards cross-generation improvement.

3.3 GRPO Objective for Skill Evolution

We optimize π_θ with a clipped GRPO surrogate that aggregates the bi-level advantage across all generations. Let $\rho_g = \pi_\theta(s_g \mid x, \mathcal{H}_{g-1}) / \pi_{\theta_{\text{old}}}(s_g \mid x, \mathcal{H}_{g-1})$ be the importance ratio between the current and data-collection policies. The **Skill-R1** objective is

$$\begin{aligned}\mathcal{L}_{\text{Skill-R1}}(\theta) = \mathbb{E}_{x\sim p(x)}\Bigg[&\sum_{g=1}^{G}\sum_{i=1}^{K} \min\Big(\rho_g\, A(y_g^{(i)}),\ \text{clip}(\rho_g,\, 1-\epsilon,\, 1+\epsilon)\, A(y_g^{(i)})\Big) \\ &- \beta\, \text{KL}\big(\pi_\theta(\cdot \mid x, \mathcal{H}_{g-1}) \,\|\, \pi_{\text{ref}}(\cdot \mid x, \mathcal{H}_{g-1})\big)\Bigg],\end{aligned} \tag{11}$$

where $A(y_g^{(i)})$ is the bi-level advantage from Equation (10), $\epsilon > 0$ is the clipping radius, and $\beta \geq 0$ weights a KL penalty anchoring the policy near a reference π_{ref}. Summing over g

generations couples the objective across the entire recurrent process in Equation (6), and thus π_θ is trained to produce improving skill trajectories rather than isolated single-step revisions. Because only π_θ is updated, the task LLM π_{task} remains frozen and can be open- or closed-source and no gradients through π_{task} are required. The full procedure is summarized in Algorithm 1.

4 Experiments

Our experiments are designed to address three key questions: (1) whether conditioning a frozen task language model on progressively evolving skills yields measurable gains over a no-skill baseline, (2) whether a multi-generation rollout framework provides benefits beyond standard GRPO training (Shao et al., 2024), and (3) whether a trained skill editor outperforms an inference-only editor without gradient-based optimization.

4.1 Experimental Setup

Tasks. We evaluate Skill-R1 on two benchmarks requiring multi-step reasoning, tool use, and verifiable answers: GAIA (Mialon et al., 2024), which consists of 165 real-world tasks over heterogeneous sources (e.g., PDFs, spreadsheets, and web pages), and WebWalker (Wu et al., 2025a), which focuses on multi-hop web navigation and cross-page reasoning. Together, they capture both structured reasoning and interactive decision-making.

Baselines. Our baselines are designed to isolate key components of the pipeline. No Skills measures the base model's standalone capability without skill conditioning. Vanilla GRPO applies standard Group Relative Policy Optimization to train the editor without bi-level advantage decomposition or multi-generation rollouts, evaluating whether conventional RL alone suffices to improve skill quality.

Skill initialization. Base skills are constructed via a two-stage distillation procedure applied per benchmark. First, GPT-4o-mini is run on a seed set of ten tasks to collect reasoning traces, including both successful and failed trajectories. Second, these traces are provided to a stronger LLM (Claude Opus 4.6), which abstracts recurring patterns into concise, reusable skill guides. For reproducibility, each Skill-R1 run uses an isolated copy of the skill directory, while baselines share a read-only version.

Training vs. Inference Decomposition. Skill-R1 (Inference) runs the full multi-generation rollout pipeline but uses a frozen Qwen editor, thereby isolating the effect of the rollout framework from any gradient-based learning. Skill-R1 (GRPO) represents the full system, where the Qwen-based editor is trained online with GRPO and bi-level advantages, capturing the additional gains from learned skill refinement.

All experiments use GPT-4o-mini as the task-solving model.

4.2 Results for GAIA

Method	**Level 1** ($n=53$)	**Level 2** ($n=86$)	**Level 3** ($n=26$)	**Overall** ($n=165$)
GPT-4o-mini (no skills)	4/53 (7.5%)	6/86 (7.0%)	0/26 (0.0%)	10/165 (6.1%)
Vanilla GRPO (Qwen3-4b)	21/53 (39.6%)	24/86 (27.9%)	4/26 (15.4%)	49/165 (29.7%)
Skill-R1 (Inference, Qwen3-4b)	22/53 (41.5%)	21/86 (24.4%)	8/26 (30.8%)	51/165 (30.9%)
Skill-R1 (GRPO, Qwen3-4b)	**31/53 (58.5%)**	**28/86 (32.6%)**	**10/26 (38.5%)**	**69/165 (41.8%)**

Table 1: Main results on the GAIA benchmark (165 tasks).

Tables 1 presents the results on GAIA. GPT-4o-mini (no skills) performs poorly, achieving only 6.1% accuracy, highlighting its difficulty with multi-step reasoning over heterogeneous sources. Vanilla GRPO substantially improves performance to 29.7%, with gains across all difficulty levels, but remains limited on more complex tasks, particularly Level 3 (15.4%),

indicating insufficient ability to compose and reuse structured knowledge. Skill-based reasoning further improves performance. The inference-only Skill-R1 setup achieves 30.9%, suggesting that the multi-generation rollout framework itself provides benefits beyond standard RL. Training the editor yields a larger improvement to 41.8%, a +12.1 point gain over Vanilla GRPO. The improvements are most prominent on harder tasks. Level 3 accuracy increases to 38.5%, compared to 0.0% for the no-skill baseline and 15.4% for Vanilla GRPO. This highlights the importance of learned skill refinement in enabling compositional reasoning over complex problems.

4.3 Results for WebWalker

Method	**Easy** ($n=8$)	**Medium** ($n=68$)	**Hard** ($n=24$)	**Overall**
GPT-4o-mini (no skills)	0/8 (0.0%)	1/68 (1.5%)	1/24 (4.2%)	2/100 (2.0%)
Vanilla GRPO (Qwen3-4b)	1/8 (12.5%)	15/68 (22.1%)	6/24 (25.0%)	22/100 (22.0%)
Skill-R1 (Inference, Qwen3-4B)	0/8 (0.0%)	14/68 (20.6%)	5/24 (20.8%)	19/100 (19.0%)
Skill-R1 (GRPO, Qwen3-4B)	**1/8 (12.5%)**	**20/68 (29.4%)**	5/24 (20.8%)	**26/100 (26.0%)**

Table 2: Results on the WebWalker benchmark (100 tasks) by difficulty.

Method	**Multi-source** ($n=63$)	**Single-source** ($n=37$)	**Overall**
GPT-4o-mini (no skills)	1/63 (1.6%)	1/37 (2.7%)	2/100 (2.0%)
Vanilla GRPO (Qwen3-4b)	14/63 (22.2%)	8/37 (21.6%)	22/100 (22.0%)
Skill-R1 (Inference, Qwen3-4B)	11/63 (17.5%)	8/37 (21.6%)	19/100 (19.0%)
Skill-R1 (GRPO, Qwen3-4B)	**18/63 (28.6%)**	8/37 (21.6%)	**26/100 (26.0%)**

Table 3: Results on the WebWalker benchmark (100 tasks) by question type.

Tables 2 and 3 present the results on WebWalker. GPT-4o-mini (no skills) achieves only 2.0% accuracy, reflecting the difficulty of multi-hop web navigation and cross-page reasoning. Vanilla GRPO improves performance to 22.0%, with consistent gains across difficulty levels. However, performance remains constrained in settings requiring deeper information synthesis, particularly in medium and multi-source tasks. Skill-based reasoning further enhances performance. The inference-only Skill-R1 setup achieves 19.0%, showing competitive performance without any gradient-based learning, while the GRPO-trained editor improves results to 26.0%, yielding a +4.0 percentage point gain over Vanilla GRPO. Improvements are especially evident in medium (29.4%) and multi-source (28.6%) settings, indicating that learned skill refinement enables more effective navigation and aggregation of information across multiple pages. An interesting observation is the relatively lower accuracy on the easy subset. This is primarily due to its small size and sensitivity to brittle navigation failures, such as incomplete page retrieval. As a result, minor errors disproportionately affect performance, leading to higher variance compared to other difficulty levels.

5 Analysis

5.1 Reward and Accuracy Progression Across Generations

To better understand the dynamics of skill evolution, we analyze both the per-task running mean reward and running accuracy across generations (Figure 2). The reward curve reflects the average verifier score over rollout populations, while the accuracy curve indicates whether at least one rollout successfully solves a task. Although rewards are binary in our setting, the two metrics capture different aspects of performance. Reward reflects the consistency of successful rollouts within a task, whereas accuracy measures only whether a task is solved at least once. As a result, the two trends are correlated but not identical, providing complementary views of both capability and reliability.

The two figures reveal a clear and largely monotonic ordering after the initial warm-up: Generation 5 outperforms Generation 4, which outperforms Generation 3, and so on, but

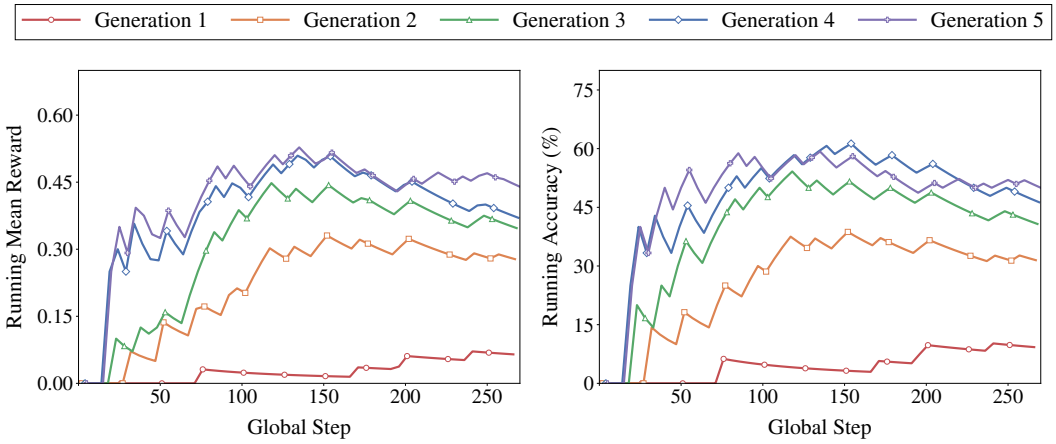

Figure 2: Accuracy and reward curves across 5 generations.

the magnitude of improvement is not uniform across generations. The largest gains occur between Generations 1 and 3, where both reward and accuracy increase sharply as useful skills begin to transfer across tasks. By the end, Generation 5 reaches a running mean reward of 0.44 and accuracy of 50.0%, compared to 0.06 reward and 9.3% accuracy for Generation 1, while Generation 4 achieves 0.37 and 46.3%. This pattern suggests that most of the broad performance gains are realized by the first three generations, while Generations 4 and 5 increasingly plateau and primarily refine an already strong skill set rather than opening up a comparably new level of capability.

Comparing reward and accuracy further clarifies this behavior. The larger improvement in reward than accuracy from Generation 4 to 5 indicates that later generations improve consistency rather than merely solving additional tasks. Once a task becomes solvable, they increase the likelihood that sampled rollouts succeed, enhancing reliability within each task. The largest jump still occurs between the early generations, especially from Generation 1 to Generations 2 and 3, indicating that the highest-value skill repairs are discovered early. Later generations improve more modestly, but the accuracy panel shows that those improvements remain meaningful rather than cosmetic. This suggests that a small number of editing rounds captures the majority of improvements, with additional iterations yielding incremental but meaningful gains in robustness and coverage.

5.2 Qualitative Analysis

5.2.1 Trained v/s Inference Skill Editor

The performance gap between Skill-R1 (GRPO) and Skill-R1 (Inference) in Tables 1 and 2 reflects a fundamental difference in how each editor interprets rollout evidence. The trained editor treats rollouts as signals for targeted repair, while the inference-only editor tends to overfit to incidental context.

Using the pdb skill as a case study, the trained editor preserves the core executable structure and introduces focused safeguards around genuine failure modes (e.g., missing files or dependencies). In contrast, the inference-only editor rewrites the skill around spurious details from sampled rollouts, often discarding essential components such as executable code. This leads to brittle, task-specific artifacts that fail to generalize.

This qualitative difference is reflected in performance, while inference-only Skill-R1 provides modest gains over Vanilla GRPO on GAIA (30.9% vs. 29.7%), it underperforms on WebWalker (19.0% vs. 22.0%), the GRPO-trained editor consistently improves performance across benchmarks (41.8% on GAIA and 26.0% on WebWalker), demonstrating that reward driven editing better captures transferable structure.

Overall, training enables selective retention of useful behaviors while suppressing noise, leading to more robust and generalizable skill updates. Full skill listings for both the GRPO-edited and inference-edited `pdb` skill (alongside the original) are provided in Section A.1.

5.2.2 *Editing Trajectory Across Generations*

Skill evolution follows a structured progression rather than simple accumulation. Early generations yield the largest gains by correcting high-impact errors, as also reflected in the sharp improvements from Generation 1 to 3 in Figure 2. Later generations provide smaller but consistent improvements by refining reliability and stabilizing behavior.

Qualitatively, this corresponds to a prune then specialize dynamic. Initial skills, which resemble broad reference documents, are quickly compressed into shorter executable routines. Subsequent generations focus on recurring failure modes and introduce tighter constraints and guardrails, resulting in compact, deployment oriented policies.

This pattern aligns with the quantitative trends: most performance gains are realized early, while later iterations primarily improve consistency rather than expanding capability. Together, these observations suggest that multi-generation evolution is effective not only for discovering better skills, but also for making them more reliable under repeated execution.

6 Related Work

6.1 Agent Skill in Language Models

Recent work treats agent skills as reusable procedural abstractions that guide LLM behavior without modifying model parameters. Surveys ((Xu & Yan, 2026), (Jiang et al., 2026c)) characterize skills as structured intermediates between tool usage and agent coordination. On the acquisition side, prior work has focused on constructing and organizing skill libraries (Nguyen et al., 2025; Huang et al., 2025a). SkillWeaver((Zheng et al., 2025)) enables agents to autonomously discover and distill skills through web interaction, while ASI((Wang et al., 2025d)) shows that programmatic skill representations improve reliability via verifiability. Similarly, CUA-Skill (Chen et al., 2026b) introduces a repository of parameterized execution graphs for computer-use agents. Adjacent agentic-LLM work treats reusable procedures and contextual conditioning as central design tools for tool use, document reasoning, and multimodal interaction (Xia et al., 2025b; Wu et al., 2025c;e; Wang et al., 2025b; Huang et al., 2025b). Other works study orchestration and system-level design (Li et al., 2026), which explores ecosystem-scale skill coordination. More recent efforts move beyond static skill banks toward skill evolution. MemSkill (Zhang et al., 2026) models memory operations as skills refined through iterative control loops, while Memento-Skills (Zhou et al., 2025) enables continual skill improvement through reflective updates without modifying the base model. However, these approaches largely rely on heuristic refinement or implicit feedback signals.

6.2 Reinforcement Learning for Language Models

Reinforcement learning has become a central paradigm for aligning language models with task objectives and feedback signals. Methods such as PPO (Schulman et al., 2017) and its variants optimize policies using scalar rewards, while more recent approaches like GRPO (Shao et al., 2024) introduce group-relative comparisons to stabilize training and improve sample efficiency. Recent variants further extend GRPO to weakly-supervised and reward-conditioned settings (Mundada et al., 2026; Zhong et al., 2026). Several works extend RL to agent settings and skill learning. SkillRL (Xia et al., 2026) jointly learns hierarchical skills and policies through recursive reinforcement learning, enabling agents to acquire reusable behaviors over time. Despite these advances, most existing methods focus on optimizing the task model itself or operate within a single-generation setting, limiting their ability to capture iterative improvement.

7 Conclusion

We presented Skill-R1, a framework for recurrent skill evolution that improves agent performance without modifying the underlying task model. By decoupling execution from skill refinement and introducing a bi-level optimization objective, Skill-R1 enables iterative, reward-driven improvement over multiple generations. Empirically, Skill-R1 consistently outperforms both no-skill baseline and standard GRPO across benchmarks, with the largest gains observed on complex, multi-step tasks. Analysis shows that early generations drive most capability improvements, while later iterations enhance reliability and consistency, highlighting the importance of both evolution and stabilization. Overall, our results suggest that skill evolution is a practical alternative to model-level adaptation, offering a lightweight yet effective mechanism for improving agent behavior in both open- and closed-source settings. **LLM usage disclosure:** AI tools were used to assist creating illustrative figures. [1]

References

Shiqi Chen, Jingze Gai, Ruochen Zhou, Jinghan Zhang, Tongyao Zhu, Junlong Li, Kangrui Wang, Zihan Wang, Zhengyu Chen, Klara Kaleb, et al. Skillcraft: Can llm agents learn to use tools skillfully? *arXiv preprint arXiv:2603.00718*, 2026a.

Tianyi Chen, Yinheng Li, Michael Solodko, Sen Wang, Nan Jiang, Tingyuan Cui, Junheng Hao, Jongwoo Ko, Sara Abdali, Leon Xu, et al. Cua-skill: Develop skills for computer using agent. *arXiv preprint arXiv:2601.21123*, 2026b.

Huan-ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Yiran Wu, et al. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence. *arXiv preprint arXiv:2507.21046*, 2025.

Chengkai Huang, Hongtao Huang, Tong Yu, Kaige Xie, Junda Wu, Shuai Zhang, Julian Mcauley, Dietmar Jannach, and Lina Yao. A survey of foundation model-powered recommender systems: From feature-based, generative to agentic paradigms. *arXiv preprint arXiv:2504.16420*, 2025a.

Chengkai Huang, Junda Wu, Yu Xia, Zixu Yu, Ruhan Wang, Tong Yu, Ruiyi Zhang, Ryan A. Rossi, Branislav Kveton, Dongruo Zhou, Julian McAuley, and Lina Yao. Towards agentic recommender systems in the era of multimodal large language models, 2025b. URL https://arxiv.org/abs/2503.16734.

Chengkai Huang, Junda Wu, Zhouhang Xie, Yu Xia, Rui Wang, Tong Yu, Subrata Mitra, Julian McAuley, and Lina Yao. Pluralistic off-policy evaluation and alignment. *arXiv preprint arXiv:2509.19333*, 2025c.

Guanyu Jiang, Zhaochen Su, Xiaoye Qu, et al. Xskill: Continual learning from experience and skills in multimodal agents. *arXiv preprint arXiv:2603.12056*, 2026a.

Pengcheng Jiang, Jiacheng Lin, Zhiyi Shi, Zifeng Wang, Luxi He, Yichen Wu, Ming Zhong, Peiyang Song, Qizheng Zhang, Heng Wang, Xueqiang Xu, Hanwen Xu, Pengrui Han, Dylan Zhang, Jiashuo Sun, Chaoqi Yang, Kun Qian, Tian Wang, Changran Hu, Manling Li, Quanzheng Li, Hao Peng, Sheng Wang, Jingbo Shang, Chao Zhang, Jiaxuan You, Liyuan Liu, Pan Lu, Yu Zhang, Heng Ji, Yejin Choi, Dawn Song, Jimeng Sun, and Jiawei Han. Adaptation of agentic ai: A survey of post-training, memory, and skills, 2026b. URL https://arxiv.org/abs/2512.16301.

Yanna Jiang, Delong Li, Haiyu Deng, Baihe Ma, Xu Wang, Qin Wang, and Guangsheng Yu. Sok: Agentic skills–beyond tool use in llm agents. *arXiv preprint arXiv:2602.20867*, 2026c.

[1] Tong and Ryan contributed to the conceptual and methodological design and did not process, store, or direct the use of project models or data.

Branislav Kveton, Xintong Li, Julian McAuley, Ryan Rossi, Jingbo Shang, Junda Wu, and Tong Yu. Active learning for direct preference optimization. *arXiv preprint arXiv:2503.01076*, 2025.

Hao Li, Chunjiang Mu, Jianhao Chen, Siyue Ren, Zhiyao Cui, Yiqun Zhang, Lei Bai, and Shuyue Hu. Organizing, orchestrating, and benchmarking agent skills at ecosystem scale. *arXiv preprint arXiv:2603.02176*, 2026.

Xintong Li, Chuhan Wang, Junda Wu, Rohan Surana, Tong Yu, Julian McAuley, and Jingbo Shang. Importance sampling for multi-negative multimodal direct preference optimization. *arXiv preprint arXiv:2509.25717*, 2025a.

Xintong Li, Junda Wu, Tong Yu, Rui Wang, Yu Wang, Xiang Chen, Jiuxiang Gu, Lina Yao, Julian McAuley, and Jingbo Shang. Commit: Coordinated multimodal instruction tuning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 11533–11547, 2025b.

Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: a benchmark for general ai assistants. In *International Conference on Learning Representations*, volume 2024, pp. 9025–9049, 2024.

Gagan Mundada, Zihan Huang, Rohan Surana, Sheldon Yu, Jennifer Yuntong Zhang, Xintong Li, Tong Yu, Lina Yao, Jingbo Shang, Julian McAuley, et al. Ws-grpo: Weakly-supervised group-relative policy optimization for rollout-efficient reasoning. *arXiv preprint arXiv:2602.17025*, 2026.

Dang Nguyen, Jian Chen, Yu Wang, Gang Wu, Namyong Park, Zhengmian Hu, Hanjia Lyu, Junda Wu, Ryan Aponte, Yu Xia, et al. Gui agents: A survey. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 22522–22538, 2025.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Zeyi Sun, Ziyu Liu, Yuhang Zang, Yuhang Cao, Xiaoyi Dong, Tong Wu, Dahua Lin, and Jiaqi Wang. Seagent: Self-evolving computer use agent with autonomous learning from experience. *arXiv preprint arXiv:2508.04700*, 2025.

Rohan Surana, Junda Wu, Xintong Li, Sheldon Yu, Yiran Jenny Shen, Chuhan Wang, Tong Yu, Prithviraj Ammanabrolu, Jingbo Shang, and Julian McAuley. MASS-DPO: Multi-negative active sample selection for direct policy optimization, 2026. URL https://openreview.net/forum?id=gFtdK7pwHg.

Chien Van Nguyen, Xuan Shen, Ryan Aponte, Yu Xia, Samyadeep Basu, Zhengmian Hu, Jian Chen, Mihir Parmar, Sasidhar Kunapuli, Joe Barrow, et al. A survey on small language models. In *Proceedings of the 15th International Conference on Recent Advances in Natural Language Processing-Natural Language Processing in the Generative AI Era*, pp. 807–821, 2025.

Jianing Wang, Junda Wu, Yupeng Hou, Yao Liu, Ming Gao, and Julian McAuley. Instructgraph: Boosting large language models via graph-centric instruction tuning and preference alignment. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 13492–13510, 2024.

Jiongxiao Wang, Qiaojing Yan, Yawei Wang, Yijun Tian, Soumya Smruti Mishra, Zhichao Xu, Megha Gandhi, Panpan Xu, and Lin Lee Cheong. Reinforcement learning for self-improving agent with skill library. *arXiv preprint arXiv:2512.17102*, 2025a.

Ruoyu Wang, Junda Wu, Yu Xia, Tong Yu, Ryan A Rossi, Julian McAuley, and Lina Yao. Dice: Dynamic in-context example selection in llm agents via efficient knowledge transfer. *arXiv preprint arXiv:2507.23554*, 2025b.

Yu Wang, Xinshuang Liu, Xiusi Chen, Sean OBrien, Junda Wu, and Julian McAuley. Self-updatable large language models by integrating context into model parameters. In *International Conference on Learning Representations*, volume 2025, pp. 16961–16979, 2025c.

Zora Zhiruo Wang, Apurva Gandhi, Graham Neubig, and Daniel Fried. Inducing programmatic skills for agentic tasks. *arXiv preprint arXiv:2504.06821*, 2025d.

Jialong Wu, Wenbiao Yin, Yong Jiang, Zhenglin Wang, Zekun Xi, Runnan Fang, Linhai Zhang, Yulan He, Deyu Zhou, Pengjun Xie, et al. Webwalker: Benchmarking llms in web traversal. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 10290–10305, 2025a.

Junda Wu, Rohan Surana, Zhouhang Xie, Yiran Shen, Yu Xia, Tong Yu, Ryan A Rossi, Prithviraj Ammanabrolu, and Julian McAuley. In-context ranking preference optimization. In *Second Conference on Language Modeling*.

Junda Wu, Hanjia Lyu, Yu Xia, Zhehao Zhang, Joe Barrow, Ishita Kumar, Mehrnoosh Mirtaheri, Hongjie Chen, Ryan A Rossi, Franck Dernoncourt, et al. Personalized multimodal large language models: A survey. *arXiv preprint arXiv:2412.02142*, 2024a.

Junda Wu, Tong Yu, Xiang Chen, Haoliang Wang, Ryan Rossi, Sungchul Kim, Anup Rao, and Julian McAuley. Decot: Debiasing chain-of-thought for knowledge-intensive tasks in large language models via causal intervention. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 14073–14087, 2024b.

Junda Wu, Xintong Li, Ruoyu Wang, Yu Xia, Yuxin Xiong, Jianing Wang, Tong Yu, Xiang Chen, Branislav Kveton, Lina Yao, et al. Ocean: Offline chain-of-thought evaluation and alignment in large language models. In *International Conference on Learning Representations*, volume 2025, pp. 100570–100589, 2025b.

Junda Wu, Yu Xia, Tong Yu, Xiang Chen, Sai Sree Harsha, Akash V Maharaj, Ruiyi Zhang, Victor Bursztyn, Sungchul Kim, Ryan A Rossi, et al. Doc-react: Multi-page heterogeneous document question-answering. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 67–78, 2025c.

Junda Wu, Yuxin Xiong, Xintong Li, Yu Xia, Ruoyu Wang, Yu Wang, Tong Yu, Sungchul Kim, Ryan A Rossi, Lina Yao, et al. Mitigating visual knowledge forgetting in mllm instruction-tuning via modality-decoupled gradient descent. *arXiv preprint arXiv:2502.11740*, 8, 2025d.

Junda Wu, Yuxin Xiong, Xintong Li, Sheldon Yu, Zhengmian Hu, Tong Yu, Rui Wang, Xiang Chen, Jingbo Shang, and Julian McAuley. Ctrls: Chain-of-thought reasoning via latent state-transition. *arXiv preprint arXiv:2507.08182*, 2025e.

Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, et al. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*, 2026.

Yu Xia, Subhojyoti Mukherjee, Zhouhang Xie, Junda Wu, Xintong Li, Ryan Aponte, Hanjia Lyu, Joe Barrow, Hongjie Chen, Franck Dernoncourt, Branislav Kveton, Tong Yu, Ruiyi Zhang, Jiuxiang Gu, Nesreen K. Ahmed, Yu Wang, Xiang Chen, Hanieh Deilamsalehy, Sungchul Kim, Zhengmian Hu, Yue Zhao, Nedim Lipka, Seunghyun Yoon, Ting-Hao Kenneth Huang, Zichao Wang, Puneet Mathur, Soumyabrata Pal, Koyel Mukherjee, Zhehao Zhang, Namyong Park, Thien Huu Nguyen, Jiebo Luo, Ryan A. Rossi, and Julian McAuley. From selection to generation: A survey of LLM-based active learning. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 14552–14569, Vienna, Austria, July 2025a. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.708. URL https://aclanthology.org/2025.acl-long.708/.

Yu Xia, Yiran Jenny Shen, Junda Wu, Tong Yu, Sungchul Kim, Ryan A Rossi, Lina Yao, and Julian McAuley. Sand: Boosting llm agents with self-taught action deliberation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 3062–3077, 2025b.

Yu Xia, Junda Wu, Sungchul Kim, Tong Yu, Ryan A Rossi, Haoliang Wang, and Julian McAuley. Knowledge-aware query expansion with large language models for textual and relational retrieval. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 4275–4286, 2025c.

Renjun Xu and Yang Yan. Agent skills for large language models: Architecture, acquisition, security, and the path forward. *arXiv preprint arXiv:2602.12430*, 2026.

Yuhang Yao, Jianyi Zhang, Junda Wu, Chengkai Huang, Yu Xia, Tong Yu, Ruiyi Zhang, Sungchul Kim, Ryan Rossi, Ang Li, et al. Federated large language models: Current progress and future directions. *arXiv preprint arXiv:2409.15723*, 2024.

Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. Memskill: Learning and evolving memory skills for self-evolving agents. *arXiv preprint arXiv:2602.02474*, 2026.

Zhehao Zhang, Ryan A Rossi, Branislav Kveton, Yijia Shao, Diyi Yang, Hamed Zamani, Franck Dernoncourt, Joe Barrow, Tong Yu, Sungchul Kim, et al. Personalization of large language models: A survey. *arXiv preprint arXiv:2411.00027*, 2024.

Boyuan Zheng, Michael Y Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, et al. Skillweaver: Web agents can self-improve by discovering and honing skills. *arXiv preprint arXiv:2504.07079*, 2025.

Haitian Zhong, Jixiu Zhai, Lei Song, Jiang Bian, Qiang Liu, and Tieniu Tan. Rc-grpo: Reward-conditioned group relative policy optimization for multi-turn tool calling agents. *arXiv preprint arXiv:2602.03025*, 2026.

Huichi Zhou, Yihang Chen, Siyuan Guo, Xue Yan, Kin Hei Lee, Zihan Wang, Ka Yiu Lee, Guchun Zhang, Kun Shao, Linyi Yang, et al. Memento: Fine-tuning llm agents without fine-tuning llms. *arXiv preprint arXiv:2508.16153*, 2025.

A Appendix

A.1 PDB Skill

This appendix provides the complete content of the `pdb` skill at three points in time, as referenced in Section 5: (1) the original skill from the shared skill library, (2) the version produced by the GRPO-trained Qwen3-4B editor after three generations of evolution (run `20260320_131744_0.25`), and (3) the version produced by the untrained base Qwen3-4B model operating in inference mode (run `20260321_185812`).

A.1.1 Original `pdb` *Skill*

The original skill is a general-purpose reference covering BioPython's full API. We reproduce the header description and the Quick Start section to illustrate the skill's initial breadth; the remainder of the document (Dependencies, PDB File Format Basics, Python Libraries, Manual Parsing, Common Tasks, Quick Reference table) is analogous in scope.

```
name: pdb
description: Use this skill whenever the user wants to read, parse, analyze,
  or manipulate PDB (Protein Data Bank) files. This includes extracting atomic
  coordinates, analyzing protein structures, identifying residues, chains,
  ligands, computing distances between atoms, visualizing molecular structures,
```

```
  and converting between molecular file formats. If the user mentions a .pdb
  file or asks about protein/molecular structure data, use this skill.

# PDB (Protein Data Bank) File Processing Guide

## Quick Start

```python
from Bio.PDB import PDBParser

parser = PDBParser(QUIET=True)
structure = parser.get_structure("protein", "structure.pdb")

# Iterate through the hierarchy: Structure -> Model -> Chain -> Residue -> Atom
for model in structure:
    for chain in model:
        print(f"Chain {chain.id}: {len(list(chain.get_residues()))} residues")
```

PDB File Format Basics

PDB files are fixed-width text files. Key record types:
ATOM	Standard residue atom coordinates
HETATM	Non-standard residue (ligands, water)
SEQRES	Full sequence of residues
REMARK	Comments and metadata

[... BioPython parsing examples, distance computation, ligand extraction,
     sequence extraction, manual ATOM-record parsing, and quick-reference
     table omitted for brevity -- all preserved in the original file ...]
```

A.1.2 GRPO-Trained Editor: Edited **pdb** *Skill*

```
name: pdb
description: [unchanged from original]

# PDB (Protein Data Bank) File Processing Guide

## Quick Start

```python
from Bio.PDB import PDBParser
import numpy as np

# Parse a local PDB file path -- ensure the file exists at the specified path
parser = PDBParser(QUIET=True)

try:
    structure = parser.get_structure("structure", "5wb7.pdb")
except FileNotFoundError:
    print("Error: The PDB file was not found. Please verify the file name.")
    raise
except ImportError:
    print("Error: BioPython is not installed. pip install biopython")
    raise

# Access the first model and chain
models = list(structure)
if not models:
    raise ValueError("No models found in the PDB structure.")
model = models[0]

chains = list(model.get_chains())
if not chains:
```
```

```
    raise ValueError("No chains found in the model.")
chain = chains[0]

residues = list(chain.get_residues())
if not residues:
    raise ValueError("No residues found in the chain.")
```

Common Misuse and Error Prevention

Note: This skill is for PDB files only. If the user provides an audio
recording (e.g., .mp3), it cannot be processed here.
```

### *A.1.3 Base Inference Editor: Edited* **pdb** *Skill*

The untrained base Qwen3-4B model replaces the entire skill with the following. No executable code remains. The "Key Requirements" section mistakenly names the GAIA task UUID `7dd30055-0198-452e-8c25-f73dbe27dcb8` as an invalid PDB identifier, and the "Common Misuse" section is wholly devoted to handling a calculus homework audio recording (`Homework.mp3`) that is unrelated to PDB processing.

```
name: pdb
description: [unchanged from original]

# PDB (Protein Data Bank) File Processing Guide

## Key Requirements and Best Practices

- Always ensure the PDB ID is valid and accessible via the RCSB PDB database.
- Use the correct PDB ID (e.g., "5wb7", not
  "7dd30055-0198-452e-8c25-f73dbe27dcb8") -- the latter is a UUID or
  invalid identifier.
- This skill does NOT handle audio files. If a user provides an audio file
  (e.g., Homework.mp3) and asks for page numbers, this skill is NOT applicable.

## Common Misuse and Clarifications

- Do not use this skill to extract page numbers from audio recordings.
- Do not attempt to parse audio files as if they were PDB files.
- If a user asks about study materials from a lecture, state that the skill
  cannot process audio content.
- Always verify the nature of the input: if it is .mp3 or .wav, respond with:
  "I cannot extract page numbers from audio files."

## Example of Incorrect Use

User: "I have a file called Homework.mp3 with my professor reading the
       recommended pages. Can you tell me the page numbers?"
Correct response: "I cannot extract page numbers from audio files. This skill
  is designed for processing PDB files, not audio recordings."

[--- No code examples remain in the edited skill ---]
```

The contrast is stark. The GRPO-trained editor adds *code* that handles real failure modes—a missing file, a missing library—while preserving the entire original reference. The base inference editor removes all code and fills the skill with content copied from two specific benchmark questions, leaving it useless for any unseen PDB task.
```